Architecture Decision Records (ADR): RAG-Diary

> บันทึกการตัดสินใจเชิงสถาปัตยกรรมที่สำคัญ พร้อมบริบท ทางเลือกที่พิจารณา และผลที่ตามมา เพื่อให้ผู้ที่มาทีหลังเข้าใจได้ว่าทำไมระบบจึงเป็นเช่นนี้

1. Document Control

หัวข้อ	รายละเอียด
ชื่อเอกสาร	Architecture Decision Records
เวอร์ชัน	1.0
วันที่จัดทำ	15 ตุลาคม 2569
ผู้จัดทำ	Business Analyst / System Analyst (เจ้าของผลงาน)
สถานะ	ผ่านการตรวจทานรอบ 1 (25 ตุลาคม 2569)
เอกสารที่เกี่ยวข้อง	C4-ARCHITECTURE, DATA-MODEL, DFD-PRIVACY

2. Purpose

เอกสารนี้ตอบคำถามว่า "ตัดสินใจอะไรลงไป ทำไม และแลกกับอะไร" โดยใช้รูปแบบของ Michael Nygard ได้แก่ สถานะ บริบท การตัดสินใจ ทางเลือกที่พิจารณา และผลที่ตามมา

3. สารบัญการตัดสินใจ

รหัส	เรื่อง	สถานะ
ADR-01	แยกฝั่งหน้าและฝั่งหลังเป็นคนละส่วน	Accepted
ADR-02	ใช้ PostgreSQL ร่วมกับ pgvector แทนฐานข้อมูลเวกเตอร์เฉพาะทาง	Accepted
ADR-03	บังคับการแยกข้อมูลด้วย Row Level Security	Accepted
ADR-04	ใช้ OpenAI เป็นผู้ให้บริการ AI	Superseded โดย ADR-08
ADR-05	ใช้รูปแบบ RAG สำหรับถามตอบ	Accepted
ADR-06	เก็บ session token ไว้ฝั่งไคลเอนต์	Superseded โดย ADR-10
ADR-07	ใช้โครงสร้าง Modular Monolith	Accepted
ADR-08	เปลี่ยนผู้ให้บริการ AI เป็น Google Gemini (free tier)	Accepted
ADR-09	ปรับเป้าหมายประสิทธิภาพการค้นหาเป็น p95 ไม่เกิน 3.5 วินาที	Accepted
ADR-10	เก็บ session ใน cookie แบบ httpOnly ผ่าน same-origin proxy	Accepted

4. ADR-01 แยกฝั่งหน้าและฝั่งหลังเป็นคนละส่วน

สถานะ: Accepted

บริบท: ระบบต้องมีหน้าจอที่โต้ตอบไวและมีตรรกะธุรกิจที่ต้องคุยกับฐานข้อมูลและผู้ให้บริการ AI การรวมทุกอย่างไว้ในที่เดียวอาจทำให้ดูแลยากเมื่อโตขึ้น

การตัดสินใจ: แยกเป็นสองส่วนชัดเจน ฝั่งหน้าใช้ Next.js และฝั่งหลังใช้ Elysia โดยคุยกันผ่าน API

ทางเลือกที่พิจารณา: (ก) ทำเป็นแอปเดียวแบบ full-stack รวมเซิร์ฟเวอร์ใน Next.js (ข) แยกเป็นสองส่วนอย่างที่เลือก

ผลที่ตามมา: ข้อดีคือทดสอบและติดตั้งแต่ละส่วนแยกกันได้ ข้อเสียคือต้องดูแลสัญญา API ให้ตรงกันและตั้งค่า CORS

5. ADR-02 ใช้ PostgreSQL ร่วมกับ pgvector

สถานะ: Accepted

บริบท: ต้องเก็บบันทึกและดัชนีเวกเตอร์เพื่อค้นคืนเชิงความหมาย

การตัดสินใจ: ใช้ PostgreSQL ที่มีส่วนขยาย pgvector และดัชนี HNSW ในฐานข้อมูลเดียว ไม่แยกเป็นฐานข้อมูลเวกเตอร์เฉพาะทาง

ทางเลือกที่พิจารณา: (ก) ฐานข้อมูลเวกเตอร์เฉพาะทางเช่น Pinecone หรือ Qdrant (ข) pgvector ใน PostgreSQL ตามที่เลือก

ผลที่ตามมา: ข้อดีคือมีแหล่งข้อมูลเดียว ลดความซับซ้อน และใช้ RLS ร่วมได้ ข้อเสียคือประสิทธิภาพที่สเกลใหญ่อาจด้อยกว่าฐานข้อมูลเฉพาะทาง

6. ADR-03 บังคับการแยกข้อมูลด้วย Row Level Security

สถานะ: Accepted

บริบท: ข้อมูลบันทึกเป็นข้อมูลส่วนตัว ต้องมั่นใจว่าไม่มีทางอ่านข้ามผู้ใช้ได้ แม้มีข้อผิดพลาดที่ชั้นแอปพลิเคชัน

การตัดสินใจ: เปิดใช้ RLS และกำหนดนโยบายแยกตามการกระทำที่ระดับฐานข้อมูล โดยยึดเงื่อนไข auth.uid() เท่ากับ user_id

ทางเลือกที่พิจารณา: (ก) ตรวจสอบสิทธิ์เฉพาะที่ชั้น API (ข) ตรวจทั้งที่ API และฐานข้อมูลตามที่เลือก

ผลที่ตามมา: ข้อดีคือได้การป้องกันสองชั้น ข้อเสียคือคำสั่งที่รันด้วยสิทธิ์เซิร์ฟเวอร์ต้องระวังไม่ให้ข้าม RLS โดยไม่ตั้งใจ

7. ADR-04 ใช้ OpenAI เป็นผู้ให้บริการ AI

สถานะ: Superseded (แทนที่ด้วย ADR-08)

บริบท: ต้องใช้บริการสร้างเวกเตอร์ สร้างคำตอบ และสร้างภาพ

การตัดสินใจ: ใช้ OpenAI ทั้งสามด้าน ได้แก่ embeddings ขนาด 1536 มิติ โมเดลแชท และโมเดลสร้างภาพ

ทางเลือกที่พิจารณา: (ก) ใช้ผู้ให้บริการหลายรายผสมกัน (ข) ใช้ผู้ให้บริการเดียวตามที่เลือก

ผลที่ตามมา: ข้อดีคือจัดการคีย์และค่าใช้จ่ายที่เดียว ข้อเสียคือผูกกับผู้ให้บริการรายเดียวและขึ้นกับความพร้อมของบริการนั้น

8. ADR-05 ใช้รูปแบบ RAG สำหรับถามตอบ

สถานะ: Accepted

บริบท: ผู้ใช้ต้องการถามคำถามกับบันทึกของตนเอง ไม่ใช่ถามความรู้ทั่วไป

การตัดสินใจ: ใช้การค้นคืนก่อนสร้างคำตอบ โดยค้นบันทึกที่เกี่ยวข้องของผู้ใช้ แล้วป้อนเป็นบริบทให้โมเดลสร้างคำตอบ

ทางเลือกที่พิจารณา: (ก) ให้โมเดลตอบจากความจำโดยไม่ค้นคืน (ข) ใช้ RAG ตามที่เลือก

ผลที่ตามมา: ข้อดีคือคำตอบอ้างอิงจากข้อมูลจริงของผู้ใช้และลดการสร้างข้อมูลเท็จ ข้อเสียคือขึ้นกับคุณภาพของดัชนีและการค้นคืน

9. ADR-06 เก็บ session token ไว้ฝั่งไคลเอนต์

สถานะ: Superseded โดย ADR-10

บริบท: ระบบยืนยันตัวตนออก token แบบไม่เก็บสถานะ ต้องตัดสินใจว่าจะเก็บที่ใด

การตัดสินใจ: เก็บ token ไว้ฝั่งไคลเอนต์ใน localStorage และแนบไปกับทุกคำขอในหัวข้อ Authorization

ทางเลือกที่พิจารณา: (ก) เก็บใน cookie แบบ httpOnly ซึ่งอ่านด้วยสคริปต์ไม่ได้ (ข) เก็บใน localStorage ตามที่เลือก

ผลที่ตามมา: ข้อดีคือฝั่งหน้าใช้งานง่ายและเหมาะกับการพัฒนาเร็ว ข้อเสียคือเสี่ยงต่อการถูกขโมยผ่านสคริปต์ จึงควรพิจารณาย้ายไป cookie แบบ httpOnly ในรอบถัดไป

10. ADR-07 ใช้โครงสร้าง Modular Monolith

สถานะ: Accepted

บริบท: ระบบมีสามโดเมนหลัก ได้แก่ การยืนยันตัวตน การจัดการบันทึก และการประมวลผล AI

การตัดสินใจ: ทำเป็นบริการเดียวที่แบ่งเป็นโมดูลตามโดเมน ได้แก่ auth, notes และ ai

ทางเลือกที่พิจารณา: (ก) แยกเป็นไมโครเซอร์วิสหลายตัว (ข) รวมเป็นโมดูลเดียวตามที่เลือก

ผลที่ตามมา: ข้อดีคือติดตั้งและดีพลอยง่าย เหมาะกับทีมเล็ก ข้อเสียคือการขยายเป็นรายโมดูลทำได้จำกัดหากระบบโตมากในอนาคต

11. ADR-08 เปลี่ยนผู้ให้บริการ AI เป็น Google Gemini (free tier)

สถานะ: Accepted

บริบท: ผู้ให้บริการเดิมคือ OpenAI แต่เครดิตของบัญชีหมด ทำให้สร้างเวกเตอร์ คำตอบ และภาพไม่ได้ทั้งหมด ระบบจึงต้องมีผู้ให้บริการที่ใช้งานได้โดยไม่เพิ่มค่าใช้จ่ายก่อน

การตัดสินใจ: ใช้ Google Gemini ผ่านคีย์เดียวจาก Google AI Studio แบ่งงานดังนี้

- Embeddings: `gemini-embedding-001` กำหนด `outputDimensionality=1536` ให้ตรงกับคอลัมน์ `vector(1536)` เดิม ไม่ต้องแก้สคีมา
- แชตและสรุปอารมณ์: `gemini-3.5-flash` ผ่าน endpoint แบบ OpenAI-compatible
- สร้างภาพ: `gemini-3.1-flash-image` (ขึ้นกับโควตาของแผน; free tier อาจตอบ 429 แล้วระบบแจ้งว่าไม่พร้อมใช้งาน)

ทางเลือกที่พิจารณา: (ก) เติมเครดิต OpenAI ต่อ (ข) ใช้ embedding ในเครื่องผ่าน Ollama ร่วมกับ opencode-go สำหรับแชต (ค) Gemini ตามที่เลือก

ผลที่ตามมา: ข้อดีคือคีย์เดียวจบ มี free tier และ embeddings ตรงกับสคีมาเดิม ข้อเสียคือโควตาภาพจำกัด ชื่อโมเดลเปลี่ยนบ่อย (gemini-2.5-flash ถูกเลิกให้ผู้ใช้ใหม่) และยังผูกกับผู้ให้บริการรายเดียว จึงอ่านชื่อโมเดลจาก environment เพื่อสลับได้โดยไม่แก้โค้ด

12. ADR-09 ปรับเป้าหมายประสิทธิภาพการค้นหาเป็น p95 ไม่เกิน 3.5 วินาที

สถานะ: Accepted

บริบท: NFR-01 เดิมกำหนด p95 ไม่เกิน 2 วินาที ตัวเลขนี้ตั้งจากหลักปฏิบัติโดยไม่มีการวัดจริง เมื่อวัดบน production ด้วย 30 บันทึกและ 20 คำค้น ได้ p50 2.39 วินาที p95 3.16 วินาที เวลาส่วนใหญ่เกิดจากการเรียก embedding ของผู้ให้บริการภายนอกซึ่งอยู่ในทุกคำค้น

การตัดสินใจ: ปรับ NFR-01 เป็น p95 ไม่เกิน 3.5 วินาที โดยนับเวลารวมการเรียก embedding และคงเกณฑ์เตือนเมื่อเกิน 5 วินาที

ทางเลือกที่พิจารณา: (ก) คง 2 วินาทีแล้วเปลี่ยนผู้ให้บริการ embedding (ข) แคช embedding ของคำค้น (ค) ปรับเป้าหมายตามประสบการณ์ใช้งานจริง

ผลที่ตามมา: เป้าหมายสอดคล้องกับเวลาจริงและต้นทุนปัจจุบัน ผู้ใช้ยังได้รับการตอบสนองในระดับที่ยอมรับได้สำหรับการค้นไดอารี่ ข้อเสียคือไม่ท้าทายให้ปรับสถาปัตยกรรมทันที จึงต้องติดตาม p95 ต่อเนื่องและกลับมาทบทวนหากเกิน 3.5 วินาที

13. ADR-10 เก็บ session ใน cookie แบบ httpOnly ผ่าน same-origin proxy

สถานะ: Accepted

บริบท: ADR-06 เก็บ token ไว้ใน localStorage ซึ่งสคริปต์ใดก็อ่านได้ ถ้ามีช่องโหว่ XSS ผู้โจมตีจะขโมย token ไปใช้แทนผู้ใช้ได้ทันที ระบบจึงต้องย้าย token ไปที่ที่ JavaScript อ่านไม่ได้

การตัดสินใจ: เก็บ token ทั้งสองตัวใน cookie แบบ httpOnly, SameSite=Lax, Path=/ และตั้ง Secure บน production โดยให้เบราว์เซอร์คุยกับ origin ของ Next.js เท่านั้น แล้วให้ Next.js rewrite เส้นทาง /api/* ไปยังเซิร์ฟเวอร์ Elysia ทำให้ cookie เป็น first-party ไม่ต้องใช้ SameSite=None ซึ่งเบราว์เซอร์สมัยใหม่บล็อก เซิร์ฟเวอร์ยังรับหัวข้อ Authorization เป็นทางถอยสำหรับสคริปต์ และตรวจ Origin ของคำขอที่ไม่ใช่ GET เพื่อกัน CSRF เพิ่มอีกชั้น

ทางเลือกที่พิจารณา: (ก) คง localStorage ไว้ (ข) ใช้ cookie แบบ SameSite=None ข้ามโดเมนตรง ๆ (ค) ใช้ same-origin proxy ตามที่เลือก

ผลที่ตามมา: ข้อดีคือ JavaScript อ่าน token ไม่ได้ ลดผลกระทบของ XSS และไม่ต้องพึ่ง third-party cookie ข้อเสียคือเพิ่มชั้น proxy หนึ่งชั้น ทำให้ debug ยากขึ้นเล็กน้อยและทุกคำขอต้องผ่าน Next.js ฝั่งหน้าจอจึงต้องไม่ตั้ง NEXT_PUBLIC_API_URL ชี้ไปโดเมนอื่นอีก

14. References

เอกสาร	ความสัมพันธ์
C4-ARCHITECTURE	สถาปัตยกรรมที่ ADR เหล่านี้กำหนด
DATA-MODEL	ที่มาของ ADR-02 และ ADR-03
DFD-PRIVACY	เหตุผลด้านความเป็นส่วนตัวที่นำมาสู่ ADR-06